

Private Lending for Aleo

A native Aleo implementation of PWN's bilateral, oracle-free lending protocol.

APPLICANT	TIMELINE	GRANT REQUEST	DELIVERABLE
PWN DAO Foundation P2P lending live on 13 EVM mainnets	4–5 months six milestones	\$100,000 capped mainnet launch	Mainnet protocol programs · SDK · reference app

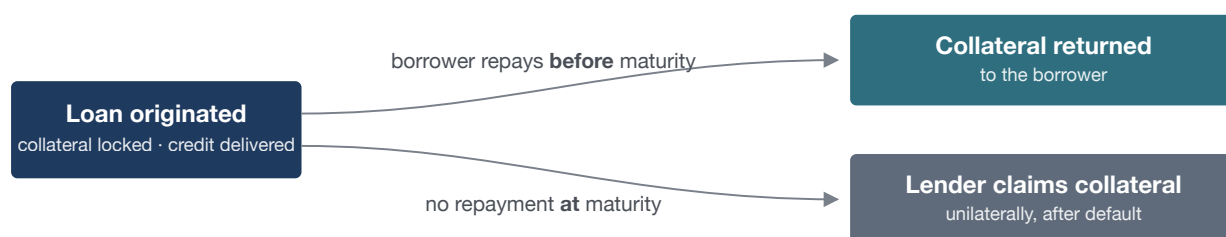
The applicant is the **PWN DAO Foundation**, which operates a peer-to-peer lending protocol live on 13 EVM mainnets. Lenders and borrowers agree on fixed loan terms directly, without price oracles or price-based liquidations. This bilateral model fits a privacy-first ZK chain particularly well, because individual loans stay isolated instead of depending on global liquidity, utilisation and liquidation state.

This grant funds a native Aleo implementation of that protocol: funded Elastic Proposals, private bilateral loans, a modular Hub and Config layer, a TypeScript SDK and a reference application — **delivered to Aleo mainnet as a usable protocol, not a prototype.**

Protocol model

Each loan fixes the credit asset and amount, the collateral asset and amount, the repayment amount and the maturity. If the borrower repays before maturity, collateral is returned. Otherwise the lender can claim it after default. There is no price feed, no health factor and no liquidation: nobody ever needs to inspect anyone else's position, which is precisely what makes the model expressible on a ZK chain where pool-based lending is not.

The Aleo implementation is a native Aleo adaptation of PWN, not a Solidity port.



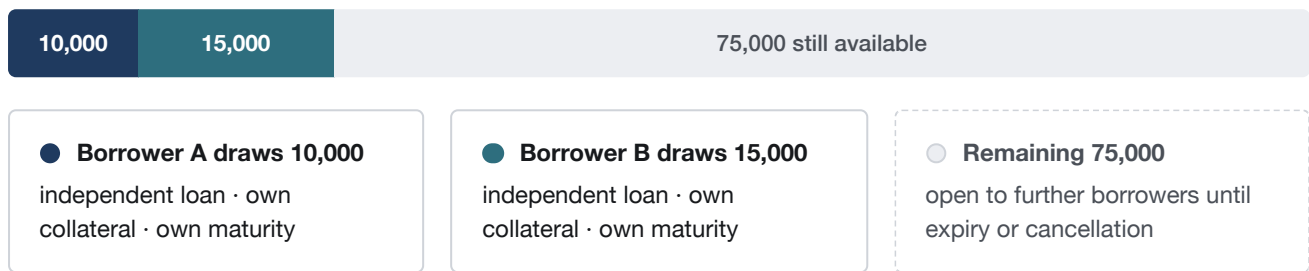
Two outcomes, decided only by the borrower's action and the clock. No third party, no price feed, no liquidation.

Elastic Proposal — the default offer type

In PWN's EVM implementation, an elastic proposal lets one lender publish a single reusable offer with an available credit limit, a minimum and maximum draw, and a fixed credit-to-collateral ratio. Multiple borrowers draw from it independently until the limit is exhausted. Each draw creates its own bilateral loan with its own collateral and maturity. It is not a pool: no shared position, no utilisation curve, no mutualised risk.

Lender commits **100,000 USDCx**

min draw **5,000** · max draw **20,000**



Why this is the default on Aleo rather than an optional extra

Aleo transactions have a single signer, and a program cannot spend a private record on a counterparty's behalf. Asynchronous acceptance therefore requires one side to commit capital in advance. Elastic proposals assume exactly that, so what is a convenience on EVM becomes the natural primitive here — and it brings three consequences we would otherwise have to engineer separately:

- every published offer is backed by real capital and cannot silently become invalid
- offers live in Aleo public state, so discovery needs no PWN backend
- multiple draws against one commitment make individual loans harder to link to a lender, so the design is also a privacy measure

Direct offers to a specific counterparty use the same mechanism with a restricted acceptor.

Discovery and indexing

Public Elastic Proposals remain canonical in Aleo public state. The SDK queries that state directly; an optional indexer or cache improves filtering and UX but is never a trusted source of truth. Private user loan records require a separate record-discovery strategy at the wallet layer.



Architecture

LAYER	PROGRAM	CONTROLLED BY	CAN AFFECT EXISTING LOANS
Governance	—	PWN DAO Foundation multisig	No
Registry & config	pwn_hub.aleo pwn_config.aleo	multisig	No — approves modules for new originations only
Proposal	pwn_elastic_proposal.aleo	lender	Own commitment only
Loan core	pwn_loan.aleo	protocol logic	Terms fixed at origination
Integration	TypeScript SDK	anyone	—
Application	Reference app, third parties	anyone	—

Governance can decide what may be created next. It can never alter, seize or redirect a loan that already exists.

Hub. Registers approved modules for new loan originations: `pwn_loan.aleo` → `ACTIVE_LOAN`, `pwn_elastic_proposal.aleo` → `PROPOSAL_MODULE`. Future installments, crowdfunding and refinancing arrive as new versioned modules.

Config. Minimal protocol-wide settings: protocol fee (zero in V1), fee recipient, multisig authority, launch caps, and an emergency stop for new originations. Loan-specific economic terms are fixed at creation and cannot be changed later through Config.

Upgradeability. Extensibility and bug remediation are separate concerns. New functionality is introduced as new Hub-approved modules; upgradeability, if enabled, is only a temporary safety mechanism for bug remediation, and custody and settlement logic capable of causing loss of funds should become immutable after audit and stabilisation.

Expected privacy model

Target V1 behaviour. Milestone 1 replaces these assumptions with measured on-chain behaviour and documents any linkability or amount leakage that remains.

PUBLIC BY DESIGN

Required for the protocol to function, not a compromise.

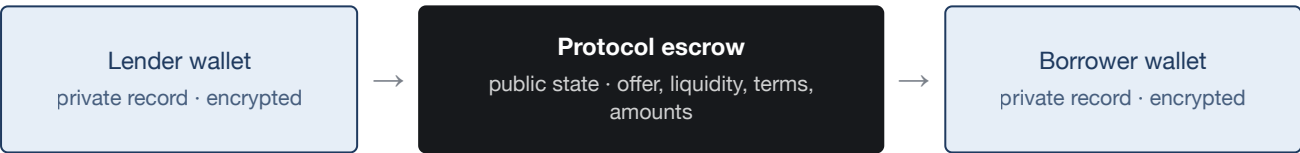
- existence of an offer
- asset pair
- available liquidity
- min and max draw
- duration
- expiry
- loan maturity
- settlement and default events

PRIVATE IN V1

Never written to protocol public state.

- identity of the lender
- identity of the borrower
- the relationship between them
- specific terms of an individual loan
- repayment history
- link between an offer and a particular borrower

WHERE THE BOUNDARY SITS



Value passes through program-controlled public state; ownership on both ends stays in encrypted records. Amounts are visible in the middle — that is the leakage measured in Milestone 1.

Known leakage — to be measured in Milestone 1

LEAK	MECHANISM	MITIGATION UNDER CONSIDERATION
Collateral amount	Custody path routes value through protocol-controlled public state	Normalised denominations; commitment-based private accounting in a later version
Draw amount	Delta in public available liquidity per draw	Normalised draw steps
Lender address	The funding transaction may be correlatable	Privately paid fees — to validate in Milestone 1
Commitment → loan link	Public by construction	Elastic proposals dilute it: twenty draws instead of one
Temporal correlation	Funding and draw close together in time	Document rather than conceal

Milestones



1	Architecture & feasibility	\$15k	2–3 weeks
----------	---------------------------------------	--------------	-----------

SCOPE Custody model, funded commitments, Hub and Config interfaces, USDCx compatibility, privacy measurement.

VALIDATION Working custody PoC; asynchronous funded commitment demonstrated; privacy matrix measured against real transactions; core data model and program interfaces agreed and documented.

2	Core protocol & loan origination	\$24k	4–5 weeks
----------	---	--------------	------------------

SCOPE Hub, Config, Elastic Proposal, direct offers, origination, collateral lock, credit delivery.

VALIDATION Funded proposal works; at least two independent draws from one commitment; direct offer works; expiry, cancellation and overdraw protections pass; Hub changes provably do not affect existing loans.

3 Repayment & default

SCOPE Repayment, collateral release, maturity handling, unilateral default claim.

VALIDATION Repayment returns collateral; early claim fails; lender can claim after maturity without borrower cooperation; double-settlement and replay tests pass.

4	SDK & integrations	\$18k 3–4 weeks
	SCOPE TypeScript SDK, public proposal discovery, record handling, USDCx integration. VALIDATION SDK supports discover, open, accept, repay and claim; public proposal discovery works without a proprietary PWN backend; USDCx integration works or the blocker is documented.	
5	Security review & audit readiness	\$15k 2–3 weeks
	SCOPE Threat model, documented invariants, adversarial test suite, freeze of custody and settlement logic, external review or scoped audit of the core loan programs. VALIDATION All critical invariants covered by tests; external reviewer engaged, or a documented audit scope and quote if audit funding sits outside this grant.	
6	Mainnet deployment & reference application	\$12k 3 weeks
	SCOPE Deploy the reviewed core programs to Aleo mainnet, publish the SDK, ship a minimal but usable lender and borrower application with selective disclosure. VALIDATION At least one real loan originated, repaid and settled on mainnet end to end by an external user; deployment addresses published; SDK available as a package; usage documented.	

Capped launch. Mainnet deployment launches with protocol-level caps on maximum commitment size and total value locked, lifted only after the security review completes. This allows real usage from day one while bounding worst-case loss.

If Milestone 1 concludes the architecture is not viable on Aleo today, we publish the findings and the privacy analysis as an open report for the ecosystem, and the remaining milestones are not claimed.

Milestone 1 feasibility questions

- 01 Can lender credit be committed once and drawn asynchronously without lender participation?
- 02 Can custody safely support private → controlled state → private asset flows?
- 03 What information is public, private, inferable or linkable in each transition?
- 04 Does the model work within USDCx compliance and token-registry constraints?
- 05 Can Proposal, Loan and token programs compose within practical proving and transaction limits?
- 06 What are the measured proving and execution times for the main flows?

Out of scope

V1 is deliberately narrow: bilateral loans with terms fixed at origination. No oracles, no price-based liquidation, no pooled lending, no bridging to PWN's EVM deployments, no KYC or credit scoring. Installments, crowdfunded funding and refinancing are planned extensions. Protocol fees are zero. V1 ships to mainnet as a usable protocol; the extensions follow later.

After the grant

The deployed protocol becomes part of the PWN DAO Foundation's maintained infrastructure and is operated alongside the existing EVM deployments, under the same governance. The SDK is published as an open-source package for third-party integrators. This is not a one-off delivery that ends when the final milestone is paid.

Possible extensions

Installment loans

Crowdfunded funding — several lenders fund one loan, each holding a private position

Refinancing

Private collateral amounts

Deeper PWN DAO governance integration

Existing references

PWN Protocol v1.5

github.com/PWNDAO/pwn_protocol

Architecture and lending-model reference: Hub, Config, loan lifecycle, Elastic Proposals, installments and crowdfunding concepts.

Arcane Finance RFQ

github.com/arcane-finance/aleo-rfq-amm-contracts

Aleo-native reference for private bilateral execution, signed offers, replay protection, expiry, and the public-liquidity / private-settlement boundary.

ProvableHQ compliant stablecoin / USDCx

github.com/ProvableHQ/compliant-stablecoin

Public and private token flows, freeze-list compliance, role administration, multisig upgrades, USDCx integration.

These are design and implementation references. Licensing and any direct code reuse are checked separately.

Open questions for the Foundation

Upgradeability

Keep selected programs upgradeable for three to six months as a bug-remediation grace period, or launch core financial logic as non-upgradeable from the start? Can upgrade authority be constrained so it cannot seize or redirect existing user funds?

Security scope

Can an external audit or a bug bounty be included as a grant milestone or budget item? Are there auditors with practical Leo experience?

Licensing

Does the Developer Grants Program require a specific open-source licence such as Apache-2.0, or is GPL-3.0 acceptable?